BEASTEST ORIGINALIUM

Réconciliation avec le cycle de vie de la réalisation du logiciel (SDLC)

Partie A - Semaines 1 à 3

Livre L3 - Collection BEASTEST

BEASTEST - Méthode d'excellence en qualité logicielle

Auteur : Léo Michel Bélanger

Illustrations et couverture : Soléane Bélanger

ORIGINALIUM

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM

MOT D'OUVERTURE

Une lettre pour toi

de Léo Michel Bélanger

Cher praticien, chère praticienne,

Tu as parcouru un chemin remarquable. Avec L2, tu as construit une vision solide et structurée du produit logiciel - ce qu'il est, ce qu'il doit accomplir, et pourquoi la qualité n'est pas une option, mais une fondation. Tu peux désormais parler du produit avec clarté et confiance. C'est une victoire qui mérite d'être célébrée.

Mais un produit logiciel ne naît pas par magie. Il emerge d'un processus - un cycle de vie complet, exigeant et fascinant - que l'industrie appelle le SDLC : le Software Development Life Cycle. C'est précisément là que ce troisième livre, L3, t'emmène.

Du concept initial jusqu'au déploiement en production et à la maintenance continue, chaque phase de ce cycle est une opportunité concrète d'ancrer la qualité. Et ce livre va te montrer, jour après jour et semaine après semaine, comment saisir chacune de ces opportunités avec intention et méthode.

Tu vas explorer les grands modèles - Waterfall, en V, itératif - avant de plonger dans l'univers Agile et ses pratiques qui ont transformé notre industrie. Ensuite, tu vas découvrir comment DevOps et le pipeline CI/CD ont révolutionné la façon dont on livre la qualité en continu.

Ce livre va transformer ta façon de voir et de vivre le développement logiciel. Tu ne seras plus un observateur du processus : tu en deviendras un acteur qualité à part entière, dès le premier jour, à chaque étape.

Fais confiance au processus. Fais confiance à ta curiosité. Et surtout, fais confiance à ta capacité de te réconcilier avec le SDLC comme l'écosystème de qualité qu'il est destiné à être.

Bonne lecture - et bonne pratique.

Léo Michel Bélanger
Fondateur, ORIGINALIUM

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM

NAVIGATION

Table des matières

Section	Contenu	Page
Mot d'ouverture	Lettre de Léo Michel Bélanger	3
SEMAINE 1 - Introduction au SDLC et intégration de la qualité dès le début		
Jour 1	Qu'est-ce que le SDLC ? Phases, objectifs et qualité	5
Jour 2	Modèles SDLC classiques - forces et faiblesses qualité	8
Jour 3	Le concept de Shift Left Testing	11
Jour 4	Phase d'idéation et d'exploration - rôle QA	14
Jour 5	Phase d'analyse des besoins - validation	17
Jour 6	Phase de conception - revues et risques qualité	20
Jour 7	Carte mentale du SDLC avec points d'ancrage qualité	23
Totalisation S1	Exercice de synthèse - Semaine 1	26
SEMAINE 2 - Les méthodologies Agile et leur impact sur la qualité		
Jour 1	Manifeste Agile et valeurs - redéfinir la qualité	28

Section	Contenu	Page
Jour 2	Scrum et qualité - rôles, cérémonies, artefacts	31
Jour 3	Kanban et qualité - flux continu et métriques	34
Jour 4	SAFe - qualité à grande échelle	37
Jour 5	DoD et DoR - standards qualité dans Agile	40
Jour 6	Backlog refinement et critères d'acceptation	43
Jour 7	Construire ton cadre d'assurance qualité Agile	46
Totalisation S2	Exercice de synthèse - Semaine 2	49
SEMAINE 3 - DevOps, CI/CD et la qualité continue		
Jour 1	Culture DevOps - collaboration et responsabilité qualité	51
Jour 2	Intégration Continue (CI) - pipelines et tests automatiques	54
Jour 3	Livraison Continue (CD) - déploiement et gates qualité	57
Jour 4	Infrastructure as Code et qualité	60
Jour 5	Sécurité dans le pipeline - DevSecOps	63
Jour 6	Métriques DevOps - DORA metrics	66
Jour 7	Concevoir ton pipeline CI/CD idéal	69
Totalisation S3	Exercice de synthèse - Semaine 3	72
Pont vers Partie B	Ce qui t'attend dans la prochaine partie	74

Section	Contenu	Page
Pages de notes	Espace libre pour tes réflexions	75

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM

SEMAINE 1 - Introduction au SDLC et intégration de la qualité dès le début

La qualité logicielle ne commence pas lors des tests. Elle commence au premier jour, à la première conversation, à la première question posée sur le produit à créer. Cette semaine, tu vas poser les fondations de cette vérité fondamentale en explorant le SDLC sous l'angle de l'excellence qualité.

Objectifs de la semaine 1

- Comprendre les phases du SDLC et leur interdépendance qualité
- Distinguer les grands modèles de développement et leurs implications
- Intérioriser le principe fondamental du Shift Left Testing
- Identifier le rôle du praticien QA dans chaque phase amont du SDLC
- Construire une carte mentale personnelle du SDLC orientée qualité

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM

Semaine 1 - Jour 1 : Qu'est-ce que le SDLC ? Phases fondamentales, objectifs et importance pour la qualité

Comprendre le cycle de vie qui donne naissance au logiciel

Le Software Development Life Cycle - ou SDLC - est le cadre structuré qui décrit toutes les activités nécessaires pour concevoir, développer, tester, déployer et maintenir un logiciel de qualité. Il ne s'agit pas d'une simple liste de tâches, mais d'un écosystème d'activités interconnectées où chaque décision influence la qualité du résultat final.

Comprendre le SDLC, c'est comprendre le terrain sur lequel tu travailles chaque jour. C'est savoir pourquoi une décision prise en phase de conception peut multiplier les défauts en phase de test - ou les prévenir complètement.

Les phases fondamentales du SDLC

Phase	Activités principales	Enjeux qualité
1. Idéation / Exploration	Définition du besoin, faisabilité, vision produit	Clarifier les attentes avant tout investissement
2. Analyse des besoins	Collecte et documentation des exigences fonctionnelles et non-fonctionnelles	Assurer la complétude, la cohérence et la testabilité des exigences
3. Conception (Design)	Architecture système, conception détaillée,	Identifier les risques architecturaux avant

Phase	Activités principales	Enjeux qualité
	modèles de données	l'écriture du code
4. Développement (Implementation)	Écriture du code, revues de code, tests unitaires	Qualité intrinsèque du code, standards de codage, couverture de tests
5. Tests (Verification)	Tests fonctionnels, de performance, de sécurité, de régression	Valider que le produit répond aux exigences et aux attentes
6. Déploiement (Release)	Mise en production, migration, configuration environnements	Livraison stable, rollback plan, validation post-déploiement
7. Maintenance	Corrections, améliorations, évolutions, monitoring	Stabilité, gestion des incidents, dette technique

Pourquoi le SDLC est central pour la qualité

Le SDLC n'est pas qu'un cadre pour les développeurs. C'est l'espace de jeu complet du praticien qualité. Chaque phase est une occasion d'agir proactivement - soit pour prévenir l'introduction de défauts, soit pour les détecter le plus tôt possible avant qu'ils ne coûtent trop cher à corriger.

La règle d'or de l'industrie est bien documentée : le coût de correction d'un défaut augmente exponentiellement selon la phase dans laquelle il est découvert. Un défaut trouvé en phase d'analyse peut coûter 1x à corriger. Le même défaut trouvé en production peut coûter 100x ou plus.

Questions de réflexion - Jour 1

1. Dans ton contexte professionnel actuel, dans quelle(s) phase(s) du SDLC passes-tu le plus de temps ? Est-ce intentionnel ou subi ?
2. As-tu déjà découvert un défaut en production qui aurait pu être évité si tu avais été impliqué plus tôt dans le processus ? Décris cette situation.
3. Selon toi, quelle est la phase du SDLC la plus souvent négligée en termes

de qualité dans ton organisation ? Pourquoi ?

4. Comment est-ce que tu te positionnes actuellement dans le cycle - testeur réactif ou acteur qualité proactif ?

Exercice pratique - Jour 1

Dessine ou décris dans l'espace ci-dessous les phases du SDLC tel que tu les vis dans ton organisation. Sont-elles toutes présentes ? Certaines sont-elles fusionnées ou absentes ? Note les points où la qualité est actuellement considérée - et ceux où elle est absente.

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM

Semaine 1 - Jour 2 : Modèles SDLC classiques - Waterfall, en V, itératif, incrémental

Connaître les modèles pour mieux choisir ton approche qualité

Il n'existe pas qu'une seule façon de traverser le SDLC. Au fil des décennies, l'industrie a développé différents modèles, chacun avec sa logique propre, ses forces et ses angles morts en matière de qualité. Aujourd'hui, tu vas les explorer avec l'oeil du praticien qualité.

Le modèle Waterfall (Cascade)

Le modèle Waterfall est le plus ancien et le plus linéaire. Les phases se succèdent de façon séquentielle et chaque phase doit être complétée avant de passer à la suivante. Pensé initialement pour des projets avec des besoins stables et bien définis, il est encore utilisé dans des domaines réglementés (aérospatial, médical, défense).

Forces qualité	Faiblesses qualité
Documentation exhaustive à chaque phase	Les tests arrivent très tard - les défauts sont coûteux à corriger
Processus prévisible et mesurable	Aucune place pour les besoins changeants ou mal compris
Jalons clairs pour la revue qualité	Le client ne voit le produit qu'en fin de cycle
Traçabilité complète exigences-tests	Accumulation de dette qualité si une phase est bâclée

Le modèle en V

Le modèle en V est une évolution du Waterfall qui fait correspondre chaque phase de développement à une phase de test miroir. La branche gauche du V représente le développement (du plus haut niveau - exigences - au plus bas - code), et la branche droite représente les tests correspondants.

Structure du modèle en V

- **Exigences systèmes** correspond aux **Tests d'acceptation**
- **Exigences logicielles** correspond aux **Tests système**
- **Conception préliminaire** correspond aux **Tests d'intégration**
- **Conception détaillée** correspond aux **Tests unitaires**
- **Implémentation** est au bas du V - point de jonction

Le modèle en V force la planification des tests dès les premières phases, ce qui est un avantage qualité majeur. Cependant, il reste séquentiel et peu adapté à des besoins évolutifs.

Le modèle itératif

Le modèle itératif divise le développement en cycles répétitifs (itérations). Chaque itération passe par toutes les phases du SDLC - de l'analyse à la mise en oeuvre - et produit une version améliorée du logiciel. Ce modèle permet d'intégrer les retours clients régulièrement et d'ajuster la direction du produit.

Le modèle incrémental

Le modèle incrémental construit le logiciel par couches successives (incréments). Chaque incrément ajoute de nouvelles fonctionnalités à un noyau de base déjà livré et fonctionnel. L'avantage est de livrer de la valeur tôt et régulièrement.

Modèle	Cycle	Flexibilité	Idéal pour	Risque qualité principal
Waterfall	Linéaire, unique	Faible	Projets stables et réglementés	Découverte tardive des défauts
En V	Linéaire avec miroir test	Faible	Systèmes critiques avec traçabilité	Planification rigide, peu adaptable
Itératif	Cycles répétitifs	Modérée	Besoins évolutifs, innovations	Accumulation de dette technique
Incrémental	Couches successives	Modérée	Livraison rapide de valeur	Intégration complexe des incréments

Questions de réflexion - Jour 2

1. Quel modèle SDLC est utilisé dans ton organisation aujourd'hui ? Est-il

explicite ou implicite ?

2. Selon toi, ce modèle est-il le mieux adapté à la nature de vos projets ? Qu'est-ce que tu changerais ?

3. Dans le modèle en V, comment est-ce que la planification des tests en amont change-t-elle ta posture en tant que praticien qualité ?

4. Quels sont les défis qualité que tu observes actuellement qui pourraient être liés au choix du modèle SDLC ?

Exercice pratique - Jour 2

Identifie un projet récent dans ton organisation. Quel modèle SDLC a-t-il suivi (même informellement) ? Quels ont été les trois principaux problèmes qualité rencontrés ? Auraient-ils été évités ou réduits avec un autre modèle ? Justifie ta réponse.

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM

Semaine 1 - Jour 3 : Le concept de Shift Left Testing

Intégrer la qualité le plus tôt possible dans le SDLC

Le Shift Left Testing est l'un des concepts les plus transformateurs de la qualité logicielle moderne. Son principe est simple mais révolutionnaire : déplacer les

activités de test et de qualité vers la gauche du SDLC - c'est-à-dire vers le début du cycle, plutôt que de les concentrer en fin de processus.

> *"Testing should not be a phase at the end of development - it should be a continuous activity woven into every stage of the SDLC."*
>
> - Principe fondamental du Shift Left Testing

Pourquoi « Shift Left » ?

Dans une représentation classique du SDLC sur une ligne horizontale, le temps s'écoule de gauche à droite. Les tests traditionnels arrivent à droite - souvent très à droite. Le Shift Left consiste à ramener ces activités qualité le plus loin possible vers la gauche, c'est-à-dire le plus tôt possible.

Les activités Shift Left concrètes

Phase SDLC	Activité Shift Left	Bénéfice qualité
Idéation	Participation aux ateliers de découverte, questionnement des hypothèses	Prévention des besoins mal définis dès la source
Analyse des besoins	Revue des exigences pour la testabilité, identification des ambiguïtés	Exigences testables, réduction des malentendus
Conception	Architecture review, revue des diagrammes, identification des risques	Problèmes architecturaux détectés avant le code
Développement	TDD (Test-Driven Development), revues de code, tests unitaires	Qualité intégrée au code dès l'écriture
Intégration	Tests d'intégration automatisés, tests de contrat	Problèmes d'interface détectés tôt

Le coût du défaut selon la phase de découverte

L'industrie a quantifié l'impact financier de la découverte tardive des défauts. Ces chiffres, issus de recherches menées notamment par le NIST et popularisés par Capers Jones, illustrent l'importance critique du Shift Left :

Phase de découverte	Coût relatif de correction
Analyse des besoins	1x (coût de référence)
Conception	3x à 6x
Développement	10x
Tests système	40x à 60x
Production (post-livraison)	100x ou plus

Shift Left vs. Shift Right - une complémentarité

Il est important de noter que le Shift Left ne signifie pas abandonner les tests tardifs. Il existe aussi le concept de Shift Right - tester en production, avec des techniques comme le canary release, le feature flagging et le monitoring avancé. Ces deux approches sont complémentaires et forment ensemble une stratégie qualité complète et continue.

Questions de réflexion - Jour 3

1. Dans ton organisation, quand commence réellement ton implication dans un projet ? Est-ce trop tard selon le principe du Shift Left ?
2. Quelles sont les résistances culturelles ou organisationnelles qui empêchent le Shift Left dans ton contexte ?
3. As-tu déjà pratiqué une revue d'exigences ou une revue de conception ? Qu'est-ce que cela t'a apporté ?
4. Comment pourrais-tu convaincre ton équipe ou ta direction d'adopter une approche Shift Left ? Quels arguments utiliserais-tu ?

Exercice pratique - Jour 3

Identifie trois activités qualité que tu pourrais commencer à faire PLUS TÔT dans ton SDLC actuel. Pour chacune, note : (1) dans quelle phase tu interviens actuellement, (2) dans quelle phase tu pourrais intervenir avec un Shift Left, (3) quel bénéfice concret tu en attendrais.

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM

Semaine 1 - Jour 4 : Phase d'idéation et d'exploration - rôle du testeur/QA dans la découverte des besoins

Ta valeur commence bien avant la première ligne de code

La phase d'idéation et d'exploration est souvent perçue comme le territoire exclusif des product managers, des analystes d'affaires et des parties prenantes. Pourtant, c'est précisément ici que ton regard critique de praticien qualité peut générer le plus de valeur - à moindre coût et avec un impact maximal.

Ce qui se passe en phase d'idéation

- Définition de la vision produit et des objectifs stratégiques
- Identification des utilisateurs cibles (personas) et de leurs besoins

- Étude de faisabilité technique et commerciale
- Exploration des contraintes réglementaires, de sécurité et de performance
- Priorisation initiale des fonctionnalités (Product Discovery)
- Identification des risques produit à haut niveau

Le rôle concret du praticien QA en idéation

Activité d'idéation	Contribution QA spécifique
Ateliers de vision produit	Poser les questions de testabilité : « Comment saurons-nous que cette fonctionnalité fonctionne correctement ? »
Définition des personas	Identifier les comportements utilisateurs limites et les cas d'usage non-nominaux
Étude de faisabilité	Évaluer la testabilité des solutions proposées et identifier les risques de validation
Exploration des contraintes	Anticiper les exigences non-fonctionnelles (performance, sécurité, accessibilité) dès la conception
Priorisation des fonctionnalités	Contribuer à l'évaluation du risque qualité de chaque fonctionnalité

Les questions puissantes du praticien QA en idéation

En phase d'idéation, ta valeur réside surtout dans ta capacité à poser les bonnes questions - celles qui révèlent les zones d'ombre avant qu'elles ne deviennent des défauts :

Questions clés à poser en phase d'idéation

- Comment mesurerons-nous le succès de cette fonctionnalité ?
- Que se passe-t-il si l'utilisateur fait exactement l'inverse de ce qu'on attend

?

- Quelles sont les données frontières (limites, valeurs extrêmes) à considérer ?
- Existe-t-il des contraintes réglementaires qui imposent des comportements spécifiques ?
- Comment cette fonctionnalité interact-elle avec les autres fonctionnalités existantes ?
- Quels sont les risques de performance à grande échelle ?
- Y a-t-il des données sensibles (PII, santé, financières) impliquées ?

De testeur à partenaire de découverte

Le passage de testeur réactif à partenaire de découverte est une transformation professionnelle majeure. En participant à l'idéation, tu ne fais pas que prévenir des défauts futurs - tu contribues à la qualité de la décision elle-même. Tu deviens un acteur de la stratégie produit, pas seulement un validateur technique.

Questions de réflexion - Jour 4

1. Es-tu actuellement invité aux ateliers d'idéation et de découverte produit ? Si non, pourquoi penses-tu que c'est le cas ?
2. Quelle question qualité, si elle avait été posée dès le début d'un projet récent, aurait évité un problème significatif ?
3. Comment peux-tu te positionner pour être inclus naturellement dans les conversations d'idéation ?
4. Quels sont tes trois axes de contribution qualité les plus uniques que tu apporterais à une session d'idéation ?

Exercice pratique - Jour 4

Pour ton prochain projet ou ta prochaine fonctionnalité en cours, prépare une liste de 10 questions de qualité que tu soumettras à l'équipe dès la phase d'idéation. Classe-les par ordre de priorité selon leur impact potentiel sur la qualité du produit.

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM

Semaine 1 - Jour 5 : Phase d'analyse des besoins - validation de la complétude et de la cohérence

Des exigences solides sont la fondation d'un produit de qualité

L'analyse des besoins est la phase où les idées se transforment en exigences documentées - fonctionnelles et non-fonctionnelles. C'est une phase critique parce que la qualité d'une exigence détermine directement la qualité du test qui la validera, et en fin de compte, la qualité du produit livré.

Les types d'exigences à valider

Type d'exigence	Description	Exemples de critères qualité
Fonctionnelles	Ce que le système doit faire	Précision, complétude, testabilité, non-ambiguïté

Type d'exigence	Description	Exemples de critères qualité
Non-fonctionnelles	Comment le système doit se comporter	Performance, sécurité, accessibilité, fiabilité, scalabilité
Contraintes	Limites imposées par l'environnement	Légales, techniques, budgétaires, temporelles
Règles d'affaires	Politiques et règles qui gouvernent le comportement	Cohérence, traçabilité, documentation

Valider la complétude d'une exigence

Une exigence est complète quand elle répond à toutes les questions nécessaires pour être testée et développée sans ambiguïté. Utilise le modèle IEEE 830 comme référence : une exigence de qualité doit être correcte, non-ambiguë, complète, cohérente, prioritisée, vérifiable, modifiable et traçable.

Le modèle INVEST pour les user stories

Dans les contextes Agile, les exigences prennent souvent la forme de user stories. Le modèle INVEST est ton outil de validation :

Lettre	Critère	Question de validation
I	Indépendante	Peut-elle être développée et testée indépendamment ?
N	Négociable	Est-elle ouverte à la discussion sur son implémentation ?
V	Valeur	Apporte-t-elle une valeur claire à l'utilisateur ou au client ?
E	Estimable	L'équipe peut-elle l'estimer en effort ?
S	Small (Petite)	Est-elle assez petite pour être réalisée en un sprint ?

Lettre	Critère	Question de validation
T	Testable	Les critères d'acceptation permettent-ils de la tester objectivement ?

Détecter les défauts d'exigences

Les défauts les plus courants dans les exigences - et comment les détecter - incluent :

- **Ambiguïté :** termes non définis, interprétations multiples possibles - Solution : exiger un glossaire et des exemples concrets
- **Incomplétion :** comportements dans des cas particuliers non définis - Solution : systématiquement questionner les cas limites et les flux d'erreur
- **Incohérence :** contradiction entre deux exigences - Solution : effectuer des revues croisées avec traçabilité
- **Non-testabilité :** « le système doit être rapide » - Solution : exiger des métriques mesurables (ex : « la page doit charger en moins de 2 secondes pour 1000 utilisateurs simultanés »)
- **Infaisabilité :** exigences techniquement impossibles ou contradictoires avec les contraintes - Solution : revue de faisabilité technique précoce

Questions de réflexion - Jour 5

1. Dans ton contexte, qui est responsable de la qualité des exigences ? Est-ce que cette responsabilité inclut le praticien QA ?
2. As-tu déjà développé ou testé une fonctionnalité basée sur une exigence ambiguë ? Quel a été l'impact ?
3. Quels outils ou techniques utilises-tu actuellement pour valider la qualité des exigences avant de commencer les tests ?

4. Comment pourrais-tu introduire une grille de validation des exigences dans ton processus actuel ?

Exercice pratique - Jour 5

Prends une exigence ou une user story de ton backlog actuel. Applique le modèle INVEST et le critère IEEE 830 pour évaluer sa qualité. Liste les lacunes trouvées et propose des améliorations concrètes. Si tu n'as pas accès à un backlog, invente une user story réaliste et applique l'exercice.

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM

Semaine 1 - Jour 6 : Phase de conception - revues de conception, architecture review, identification des risques qualité

Identifier les risques avant qu'ils ne deviennent des défauts

La phase de conception est le moment où l'équipe décide COMMENT le système va être construit. C'est une phase stratégique pour le praticien qualité, car les décisions architecturales prises ici vont influencer la testabilité, la maintenabilité, la performance et la sécurité du produit pour des années à venir.

Deux niveaux de conception à comprendre

Niveau	Ce qu'il couvre	Enjeux qualité
Conception de haut niveau (HLD)	Architecture globale, composants majeurs, interfaces systèmes, choix technologiques	Scalabilité, résilience, couplage des composants, testabilité architecturale
Conception détaillée (LLD)	Modules, classes, algorithmes, structures de données, interfaces internes	Complexité cyclomatique, couverture de tests unitaires, maintenabilité

Mener une Architecture Review efficace

Une Architecture Review n'est pas une réunion de validation formelle - c'est une conversation structurée pour identifier les risques avant qu'ils ne se concrétisent. En tant que praticien qualité, tu contribues avec une perspective unique : celle de la validation et de l'observabilité.

Checklist d'une Architecture Review orientée qualité

- Les composants sont-ils suffisamment découplés pour être testés indépendamment ?
- Les interfaces entre composants sont-elles clairement définies et documentées ?
- L'architecture permet-elle l'injection de données de test (testabilité) ?
- Les mécanismes de logging et de monitoring sont-ils prévus dès la conception ?
- Les exigences de performance ont-elles été traduites en contraintes architecturales ?
- Les Single Points of Failure (SPOF) ont-ils été identifiés et mitigés ?
- La sécurité (authentification, autorisation, chiffrement) est-elle intégrée dès

la conception ?

- La gestion des erreurs et des exceptions est-elle définie à chaque niveau ?

Identifier les risques qualité en conception

Les risques qualité en phase de conception peuvent être classés selon leur nature :

- **Risques de performance :** architecture non scalable, goulots d'étranglement identifiables, dépendances synchrones non nécessaires
- **Risques de sécurité :** données sensibles non chiffrées, authentification insuffisante, surfaces d'attaque non protégées
- **Risques de maintenabilité :** forte dépendance entre modules, code non modulaire, absence de patterns reconnus
- **Risques de testabilité :** composants non injectables, logique métier enfouie dans l'interface, absence de mock/stub points
- **Risques d'intégration :** APIs mal définies, formats de données incompatibles, gestion des versions inconsistante

Questions de réflexion - Jour 6

1. Es-tu actuellement impliqué dans les revues de conception ou d'architecture dans ton organisation ? Sous quelle forme ?
2. Quelle décision architecturale récente a eu un impact négatif sur la testabilité ou la qualité du produit ? Aurait-elle pu être différente avec une revue QA ?
3. Comment documentes-tu les risques qualité identifiés en phase de conception ? Existe-t-il un registre de risques dans ton équipe ?
4. Quelles compétences techniques dois-tu développer pour contribuer plus

efficacement aux Architecture Reviews ?

Exercice pratique - Jour 6

Prends un diagramme d'architecture ou une description technique d'un système que tu connais. Applique la checklist d'Architecture Review orientée qualité. Identifie au moins cinq risques qualité et propose une action de mitigation pour chacun. Note tes observations ci-dessous.

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM

Semaine 1 - Jour 7 : Créer ta carte mentale du SDLC avec les points d'ancrage qualité

Synthétiser une semaine de découverte en une vision personnelle

Tu arrives au septième jour de ta première semaine. Tu as exploré les fondations du SDLC, les modèles classiques, le Shift Left Testing, et le rôle du praticien qualité dans les phases amont. Il est maintenant temps de synthétiser tout cela en un outil visuel puissant : ta carte mentale du SDLC avec les points d'ancrage qualité.

Qu'est-ce qu'une carte mentale du SDLC orientée qualité ?

C'est ta représentation personnelle du cycle de vie du développement logiciel, augmentée des points clés où tu interviens, où tu poses des questions, où tu

préviens et où tu détectes. C'est un outil de référence vivant qui évoluera tout au long de ta carrière.

Structure suggérée de ta carte mentale

Noeud central	Branches principales	Sous-branches qualité
SDLC Qualité	Idéation	Questions QA, risques produit, personas, faisabilité qualité
Analyse des besoins	INVEST, IEEE 830, testabilité, ambiguïtés, cohérence	
Conception	Architecture Review, risques qualité, testabilité architecturale	
Développement	TDD, revues de code, tests unitaires, standards	
Tests	Stratégie, types de tests, niveaux, outils, reporting	
Déploiement	Smoke tests, validation post-déploiement, rollback	
Maintenance	Monitoring, gestion des incidents, dette technique	

Les 5 points d'ancrage qualité à ne jamais oublier

- **Point d'ancrage 1 - Idéation :** « Est-ce que je comprends et questionne ce qu'on veut construire ? »
- **Point d'ancrage 2 - Besoins :** « Est-ce que chaque exigence est testable, complète et cohérente ? »
- **Point d'ancrage 3 - Conception :** « Est-ce que l'architecture permet de valider ce qui sera construit ? »
- **Point d'ancrage 4 - Développement :** « Est-ce que la qualité est intégrée au code dès l'écriture ? »
- **Point d'ancrage 5 - Production :** « Est-ce que je surveille la qualité réelle vécue par l'utilisateur ? »

Exercice pratique - Jour 7 : Ta carte mentale personnelle

Crée ta carte mentale du SDLC avec tes points d'ancrage qualité personnels. Tu peux la dessiner à la main sur une feuille séparée, utiliser un outil numérique (Miro, MindMeister, XMind), ou la décrire textuellement dans l'espace ci-dessous. L'important est qu'elle reflète TON contexte, TON rôle et TES opportunités d'impact qualité. Elle doit être suffisamment personnelle pour que tu puisses la partager avec ton équipe lors d'une prochaine rétrospective.

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM

EXERCICE DE TOTALISATION - SEMAINE 1

TTLSṬ~ - Semaine 1

Dimension 1 - Substantif linguistique

« Action d'additionner ; action de considérer l'ensemble, le total, de synthétiser. »

Dimension 2 - Philosophie ORIGINALIUM

« Rassembler et réaliser totalement dans son ensemble - avant, pendant et après - en continu, sans interruption. »

Totaliser ta compréhension du SDLC comme un tout

La totalisation n'est pas un résumé passif. C'est un acte actif de synthèse où tu rassembles toutes les pièces que tu as explorées cette semaine pour les voir comme un tout cohérent et vivant. Le SDLC n'est pas une liste de phases - c'est un écosystème de qualité où chaque partie influence le tout.

Questions de totalisation - Semaine 1

1. **Totalise le SDLC :** Si tu devais expliquer le SDLC comme un organisme vivant à un collègue non-technique, comment le décrirais-tu ? Quels sont ses organes vitaux, ses flux, ses points de vulnérabilité ?
2. **Totalise ta position :** Où te situes-tu aujourd'hui dans ce SDLC ? Où est ton impact maximal ? Où as-tu des zones aveugles à développer ?
3. **Totalise le Shift Left :** Comment le principe du Shift Left transforme-t-il concrètement ta façon de te voir en tant que praticien qualité ? Qu'est-ce que tu vas faire différemment dès demain ?
4. **Totalise la valeur :** Quelles sont les trois actions les plus impactantes que tu pourrais prendre dans les 30 prochains jours pour améliorer la qualité de ton SDLC actuel ?
5. **Totalise le tout :** Comment chaque phase du SDLC que tu as étudiée cette semaine contribue-t-elle à la qualité du produit final ? Quelle est la connexion essentielle entre idéation et maintenance ?

Engagement de totalisation - Semaine 1

Complète cette phrase dans l'espace ci-dessous : « Après cette première semaine, je comprends maintenant que ma valeur en tant que praticien qualité dans le SDLC est... et mon engagement pour la semaine prochaine est de... »

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM

SEMAINE 2 - Les méthodologies Agile et leur impact sur la qualité

L'Agile n'est pas une méthode de développement. C'est une philosophie de collaboration et d'adaptation continue qui a profondément transformé la façon dont les équipes créent des logiciels de qualité. Cette semaine, tu vas explorer ses fondements, ses cadres pratiques et son impact direct sur ton travail de praticien qualité.

Objectifs de la semaine 2

- Comprendre les valeurs et principes du Manifeste Agile et leur lien avec la qualité
- Maîtriser Scrum, Kanban et SAFe comme cadres d'assurance qualité
- Définir et appliquer la Definition of Done et la Definition of Ready
- Maîtriser le backlog refinement et les critères d'acceptation
- Construire un cadre d'assurance qualité Agile adapté à ton équipe

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM

Semaine 2 - Jour 1 : Manifeste Agile et valeurs - comment l'agilité redéfinit la qualité logicielle

Les racines philosophiques d'une révolution qualité

En 2001, dix-sept praticiens du développement logiciel se sont réunis à Snowbird, Utah, pour partager leurs expériences et leurs convictions. Ils ont signé le Manifeste pour le Développement Agile de Logiciels - un texte court mais révolutionnaire qui a changé l'industrie pour toujours.

Les 4 valeurs fondamentales du Manifeste Agile

Ce que l'Agile valorise davantage	Sans nier la valeur de
Les individus et leurs interactions	Les processus et les outils
Des logiciels opérationnels	Une documentation exhaustive
La collaboration avec les clients	La négociation contractuelle
L'adaptation au changement	Le suivi d'un plan

Comment ces valeurs redéfinissent la qualité

- **Individus et interactions :** La qualité est une responsabilité partagée de toute l'équipe, pas seulement du département QA. Chaque développeur est un acteur qualité.
- **Logiciels opérationnels :** La qualité se mesure par la valeur livrée et fonctionnelle à chaque itération, pas par la complétude d'un plan de test.
- **Collaboration avec les clients :** La qualité est définie par les besoins réels des utilisateurs, validés continuellement - pas seulement par les spécifications initiales.

- **Adaptation au changement :** Les stratégies qualité doivent être flexibles et s'adapter aux nouvelles informations, aux changements de priorité et aux apprentissages du terrain.

Les 12 principes Agile sous l'angle qualité

Parmi les 12 principes du Manifeste, plusieurs ont une résonance directe avec la qualité logicielle :

- « Notre priorité est de satisfaire le client en livrant tôt et régulièrement des logiciels utiles. » - La qualité doit être présente à chaque livraison, même les premières.
- « Un logiciel opérationnel est la principale mesure d'avancement. » - La qualité de fonctionnement est la mesure de progrès, pas la documentation ou les plans.
- « Une attention continue à l'excellence technique et à une bonne conception renforce l'agilité. » - La qualité technique n'est pas optionnelle, elle est un enabler de l'agilité.
- « Les équipes réfléchissent régulièrement aux moyens de devenir plus efficaces. » - L'amélioration continue de la qualité est intégrée au processus Agile.

Questions de réflexion - Semaine 2, Jour 1

1. Comment les valeurs Agile ont-elles changé (ou pas encore changé) ta façon de travailler la qualité dans ton organisation ?
2. Est-ce que la qualité est vraiment une responsabilité partagée dans ton équipe, ou est-elle encore perçue comme la responsabilité exclusive du QA ?
3. Quels sont les principes Agile qui sont bien appliqués dans ton contexte ? Lesquels sont ignorés au détriment de la qualité ?
4. Comment pourrais-tu utiliser le Manifeste Agile comme levier pour

améliorer ta culture qualité d'équipe ?

Exercice pratique - Semaine 2, Jour 1

Pour chacune des 4 valeurs Agile, note un exemple concret de ton contexte où cette valeur est bien incarnée, et un exemple où elle est violée. Propose une action pour renforcer chaque valeur dans ton équipe.

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM

Semaine 2 - Jour 2 : Scrum et qualité - rôles, cérémonies et artefacts qualité

Scrum comme écosystème d'assurance qualité

Scrum est le cadre Agile le plus répandu dans l'industrie du logiciel. Pour beaucoup, il est perçu comme un cadre de gestion de projet. Pour toi, praticien qualité, il doit être vu comme un écosystème d'opportunités qualité intégrées à chaque rôle, chaque cérémonie et chaque artefact.

Les rôles Scrum sous l'angle qualité

Rôle Scrum	Responsabilité qualité principale
Product Owner (PO)	Définir des critères d'acceptation clairs, prioriser selon la valeur et le

Rôle Scrum	Responsabilité qualité principale
	risque qualité, valider les livrables selon les attentes des parties prenantes
Scrum Master	Éliminer les obstacles qui freinent la qualité, faciliter les cérémonies d'amélioration continue, protéger l'équipe pour qu'elle maintienne ses standards qualité
Équipe de développement	Qualité collective du code, tests unitaires et d'intégration, respect de la Definition of Done, revues de code
QA / Testeur	Stratégie de tests, exécution, automatisation, reporting, mentoring qualité de l'équipe, participation active à toutes les cérémonies

Les cérémonies Scrum comme opportunités qualité

- **Sprint Planning :** Valider que les user stories ont des critères d'acceptation testables avant d'être acceptées dans le sprint. Estimer l'effort de test. Identifier les risques qualité du sprint.
- **Daily Scrum :** Signaler les bloquants qualité, les défauts trouvés, les risques émergents. Partager les informations de test avec l'équipe.
- **Sprint Review :** Valider que chaque item livré respecte la Definition of Done. Tester en direct avec les parties prenantes si possible.
- **Sprint Retrospective :** Analyser les défauts du sprint - leurs causes, leur distribution, les leçons apprises. Proposer des améliorations du processus qualité.
- **Backlog Refinement :** Améliorer la testabilité des user stories futures. Identifier les dépendances et les risques en amont.

Les artefacts Scrum enrichis par la qualité

Artefact	Enrichissement qualité
Product Backlog	Chaque item a des critères d'acceptation, une estimation de risque qualité et une priorité influencée par le risque
Sprint Backlog	Inclut explicitement les tâches de test et les activités qualité - elles ne sont pas implicites
Increment	Doit satisfaire la Definition of Done pour être considéré comme livrable - pas de négociation sur la qualité

Questions de réflexion - Semaine 2, Jour 2

1. Dans tes cérémonies Scrum actuelles, où est-ce que la qualité est explicitement abordée ? Où est-elle ignorée ?
2. Ton Sprint Backlog inclut-il explicitement les tâches de test ? Ou les tests sont-ils implicites et souvent sous-estimés ?
3. Comment est-ce que le praticien QA participe à ton Sprint Planning ? Est-ce proactif ou réactif ?
4. Quelle amélioration qualité as-tu proposée lors de ta dernière Rétrospective ? A-t-elle été implémentée ?

Exercice pratique - Semaine 2, Jour 2

Crée une grille d'audit qualité de tes cérémonies Scrum actuelles. Pour chaque cérémonie, note : est-ce que la qualité est explicitement adressée ? Qui en parle ? Quelles décisions qualité sont prises ? Quelles opportunités qualité sont manquées ? Propose trois améliorations immédiates.

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM

Semaine 2 - Jour 3 : Kanban et qualité - flux continu, WIP limits, métriques de flux

La qualité dans un système de flux continu

Kanban est un cadre Agile différent de Scrum. Là où Scrum organise le travail en sprints itératifs, Kanban favorise un flux de travail continu, visualisé sur un tableau et régulé par des limites de travail en cours (WIP - Work In Progress). Pour le praticien qualité, Kanban offre une visibilité incomparable sur les goulots d'étranglement qualité.

Les principes Kanban à travers le prisme qualité

- **Visualiser le flux :** Le tableau Kanban rend visible l'état de chaque item, y compris les items bloqués pour des raisons qualité (défauts, revues en attente, retests).
- **Limiter le WIP :** Les limites WIP empêchent la surcharge de l'équipe, qui est l'une des principales causes de régression qualité. Une équipe surchargée fait des erreurs.
- **Gérer le flux :** Un flux fluide et stable est un indicateur de santé qualité du processus. Les interruptions de flux révèlent des problèmes systémiques.
- **Rendre les politiques explicites :** Les critères de qualité pour passer d'une colonne à l'autre doivent être explicites et respectés par tous.
- **Améliorer en collaboration :** Les données de flux alimentent des conversations d'amélioration continue orientées faits.

Les métriques de flux - tes indicateurs qualité Kanban

Métrique	Définition	Signal qualité
Lead Time	Temps entre la création de l'item et sa livraison	Un Lead Time qui s'allonge peut signaler des problèmes qualité récurrents (retests, corrections)
Cycle Time	Temps entre le début actif du travail et la livraison	Un Cycle Time élevé en phase de test indique des défauts ou des blocages qualité
Throughput	Nombre d'items livrés par unité de temps	Un throughput qui chute peut indiquer une dégradation de la qualité du code entrant en test
WIP actuel	Nombre d'items en cours simultanément	Un WIP élevé augmente les risques de défauts par manque de focus
Taux de blocage	Pourcentage d'items bloqués pour cause qualité	Indicateur direct de la fréquence et de la nature des problèmes qualité

Colonnes Kanban orientées qualité

Une configuration de tableau Kanban qui intègre explicitement la qualité pourrait inclure :

- À faire | En analyse | En développement | En revue de code | En test | En revue QA | Prêt à déployer | Livré
- Chaque transition entre colonnes doit avoir des critères explicites - c'est ton opportunity d'intégrer des standards qualité dans le flux.

Questions de réflexion - Semaine 2, Jour 3

1. Si ton équipe utilise Kanban, est-ce que tes colonnes reflètent les étapes

qualité (revue, test, validation) ? Sinon, que manque-t-il ?

2. Quels sont tes Lead Time et Cycle Time actuels ? Sais-tu combien de temps les items passent spécifiquement en test ?

3. Comment les limites WIP sont-elles définies dans ton équipe ? Tiennent-elles compte de la capacité de test ?

4. As-tu déjà analysé tes métriques de flux pour identifier des problèmes systémiques de qualité ? Qu'as-tu trouvé ?

Exercice pratique - Semaine 2, Jour 3

Dessine le tableau Kanban idéal pour ton équipe qui intègre explicitement toutes les étapes qualité. Définis les critères de passage pour chaque colonne. Propose des limites WIP pour chaque étape. Identifie où tu places tes points de contrôle qualité les plus critiques.

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM

Semaine 2 - Jour 4 : SAFe (Scaled Agile Framework) - qualité à grande échelle dans les organisations complexes

Quand l'Agile doit s'adapter à la complexité organisationnelle

Le Scaled Agile Framework - SAFe - est le cadre le plus utilisé pour appliquer les principes Agile dans des organisations de grande taille avec de multiples équipes qui travaillent en coordination sur des produits complexes. Si Scrum et Kanban opèrent au niveau de l'équipe, SAFe opère à plusieurs niveaux simultanément - et la qualité doit être assurée à chacun d'eux.

Les niveaux de SAFe et leurs enjeux qualité

Niveau SAFe	Description	Enjeux qualité spécifiques
Team Level	Équipes Agile individuelles (Scrum ou Kanban)	Qualité des user stories, Definition of Done, tests automatisés, intégration continue
Program Level (ART)	Agile Release Train - ensemble d'équipes alignées sur un Programme Increment (PI)	Cohérence qualité entre équipes, tests d'intégration inter-équipes, System Demo, Inspect and Adapt
Large Solution Level	Coordination de plusieurs ARTs pour des solutions très complexes	Tests de systèmes-de-systèmes, validation de l'architecture globale, coordination des releases
Portfolio Level	Alignement stratégique des investissements et de la gouvernance	Qualité de la vision, cohérence des Epics, métriques de qualité portfolio

Concepts SAFe clés pour le praticien qualité

- **Built-in Quality :** SAFe fait de la qualité intégrée (Built-in Quality) l'une de ses cinq valeurs fondamentales. La qualité n'est pas ajoutée après - elle est construite dans chaque étape.

- **Program Increment (PI) :** Un cycle de 8 à 12 semaines durant lequel plusieurs équipes livrent ensemble un incrément de valeur. La coordination qualité entre équipes est critique.
- **System Demo :** À la fin de chaque itération, une démonstration intégrée du travail de toutes les équipes - c'est une validation qualité systémique.
- **Inspect and Adapt (I&A) :** Cérémonie de fin de PI pour analyser les métriques, identifier les problèmes systémiques et mettre en place des améliorations - l'équivalent d'une rétrospective à grande échelle orientée qualité.

Les 5 dimensions de la Built-in Quality dans SAFe

6. **Flow :** Des flux de valeur fluides sans interruption qualité
7. **Architecture et Design :** Une base technique solide qui supporte la qualité à long terme
8. **Code :** Pratiques de qualité du code - TDD, pair programming, revues de code
9. **Système :** Tests d'intégration continue à l'échelle du système
10. **Release :** Capacité à livrer des releases de qualité à la demande

Questions de réflexion - Semaine 2, Jour 4

1. Ton organisation utilise-t-elle SAFe ou un autre cadre d'Agile à l'échelle ? Quels sont les défis qualité spécifiques à cette échelle ?
2. Comment est-ce que la cohérence qualité est assurée entre plusieurs équipes qui travaillent sur le même produit ?
3. As-tu déjà participé à un Inspect and Adapt ? Quels problèmes qualité systémiques ont été identifiés ?
4. Quels sont les risques qualité spécifiques à la coordination inter-équipes

dans ton contexte ?

Exercice pratique - Semaine 2, Jour 4

Identifie un problème qualité qui affecte plusieurs équipes dans ton organisation (ou imagine un scénario réaliste). Analyse comment ce problème se manifeste à chaque niveau (équipe, programme, portfolio). Propose une stratégie de résolution qui adresse le problème à tous les niveaux concernés.

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM

Semaine 2 - Jour 5 : Definition of Done (DoD) et Definition of Ready (DoR) - standards de qualité dans Agile

Les contrats qualité de ton équipe Agile

La Definition of Done (DoD) et la Definition of Ready (DoR) sont deux des outils qualité les plus puissants et les plus sous-utilisés dans les équipes Agile. Ensemble, ils définissent les standards de qualité qui encadrent le travail - l'un à l'entrée du sprint, l'autre à la sortie.

La Definition of Ready (DoR) - La porte d'entrée qualité

La DoR est l'ensemble des critères qu'une user story doit satisfaire AVANT de pouvoir être acceptée dans un sprint. C'est un contrat d'entrée qui protège l'équipe

contre le travail bâclé et les exigences mal définies - les principales sources de retard et de défauts.

Exemple de Definition of Ready solide

- La user story suit le format standard : En tant que [persona], je veux [action] afin de [bénéfice]
- Les critères d'acceptation sont définis, clairs et testables
- Les maquettes ou wireframes sont disponibles si pertinents
- Les dépendances techniques ont été identifiées et sont résolues ou planifiées
- La user story a été estimée en points d'effort par l'équipe
- Les risques qualité ont été identifiés et documentés
- La user story est indépendante et livrable dans un sprint
- Les cas de test de haut niveau ont été identifiés par le QA

La Definition of Done (DoD) - La porte de sortie qualité

La DoD est l'ensemble des critères qu'un item doit satisfaire AVANT d'être considéré comme terminé et potentiellement livrable. C'est l'un des mécanismes de qualité les plus importants de Scrum - et souvent le plus négocié sous la pression.

Exemple de Definition of Done robuste

- Le code est écrit et respecte les standards de codage de l'équipe
- Les tests unitaires sont écrits et passent à 100% (couverture minimale : 80%)
- Les tests d'intégration associés passent

- La revue de code a été effectuée par au moins un pair
- Les critères d'acceptation sont validés par des tests manuels ou automatisés
- Aucun défaut bloquant ou critique n'est ouvert sur cet item
- La documentation technique est mise à jour si pertinente
- Les tests de régression automatisés ont été exécutés et passent
- La fonctionnalité a été démontrée au Product Owner et approuvée
- Le code est mergé et déployé dans l'environnement de test

Niveaux de DoD dans SAFe

Dans les organisations qui utilisent SAFe, la DoD opère à plusieurs niveaux qui s'emboîtent :

- **DoD d'une user story :** Critères au niveau du code et des tests unitaires
- **DoD d'une feature :** Critères d'intégration, de test système et de documentation
- **DoD d'un Program Increment :** Critères de déploiement, de validation système complète et de release notes

Questions de réflexion - Semaine 2, Jour 5

1. Ton équipe a-t-elle une DoD explicite et documentée ? Est-elle réellement respectée ou est-elle souvent négociée sous la pression ?
2. Quels sont les items de DoD qui sont le plus souvent sacrifiés dans ton contexte ? Quel est l'impact sur la qualité à long terme ?
3. Utilises-tu une DoR dans ton équipe ? Quels problèmes cela t'aiderait-il à

éviter ?

4. Comment pourrais-tu améliorer la DoD de ton équipe pour qu'elle reflète mieux les standards qualité que tu vises ?

Exercice pratique - Semaine 2, Jour 5

Rédige une DoD et une DoR idéales pour ton équipe actuelle. Assure-toi qu'elles sont spécifiques, mesurables et réalistes. Ensuite, compare-les avec ce qui existe actuellement (ou l'absence de définition) et identifie les écarts les plus significatifs. Propose un plan pour combler ces écarts en 3 sprints.

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM

| TTLST~

Semaine 2 - Jour 6 : Backlog refinement et critères d'acceptation - bases pour des tests pertinents

La qualité des tests naît de la qualité des critères d'acceptation

Le backlog refinement (ou grooming) est la cérémonie Agile où l'équipe examine et améliore les user stories du backlog avant leur planification dans un sprint futur. C'est une cérémonie cruciale pour le praticien qualité - c'est ici que tu poses les bases de tes tests avant même que le développement ne commence.

Ton rôle dans le backlog refinement

- Identifier les ambiguïtés dans les user stories et demander des clarifications
- Évaluer la testabilité de chaque story et proposer des améliorations
- Co-créer les critères d'acceptation avec le Product Owner et les développeurs
- Identifier les risques qualité et les cas de test à haut niveau
- Détecter les dépendances qui pourraient affecter la stratégie de test
- Questionner les hypothèses et les cas limites non couverts

Critères d'acceptation de qualité - le format Given-When-Then

Le format Given-When-Then (GWT), issu du Behavior-Driven Development (BDD), est l'outil le plus efficace pour écrire des critères d'acceptation précis et directement transformables en tests :

Format Given-When-Then

ÉTANT DONNÉ QUE [contexte initial - état du système et/ou de l'utilisateur]
QUAND [action effectuée par l'utilisateur ou par le système]
ALORS [résultat attendu mesurable et observable]

Exemple :
ÉTANT DONNÉ QUE l'utilisateur est connecté et a un panier avec 3 articles
QUAND il clique sur « Passer la commande »
ALORS le système affiche un résumé de commande avec les 3 articles, le total TTC et un numéro de confirmation unique

Les 7 caractéristiques d'un bon critère d'acceptation

11. **Précis :** sans ambiguïté sur ce qui est attendu
12. **Mesurable :** on peut déterminer objectivement s'il est satisfait
13. **Réaliste :** techniquement faisable dans les contraintes du projet
14. **Complet :** couvre les flux nominaux ET les flux d'erreur

15. **Indépendant :** peut être validé sans dépendance à d'autres critères
16. **Orienté utilisateur :** décrit le comportement du point de vue de l'utilisateur, pas de l'implémentation
17. **Automatable :** peut être traduit directement en test automatisé

Questions de réflexion - Semaine 2, Jour 6

1. Participes-tu activement aux sessions de backlog refinement ? Quelle est ta contribution typique ?
2. Les critères d'acceptation dans ton backlog sont-ils actuellement dans un format structuré (GWT ou autre) ? Sont-ils testables directement ?
3. Combien de fois as-tu découvert en test qu'un critère d'acceptation était ambigu ou incomplet ? Quel a été l'impact sur le sprint ?
4. Comment peux-tu améliorer la qualité des critères d'acceptation dans ton équipe dès le prochain backlog refinement ?

Exercice pratique - Semaine 2, Jour 6

Prends une user story de ton backlog qui a des critères d'acceptation incomplets ou ambigus. Réécris-les en format Given-When-Then, en t'assurant de couvrir : (1) le flux nominal, (2) au moins deux flux alternatifs, (3) au moins deux cas d'erreur. Évalue chaque critère selon les 7 caractéristiques d'un bon critère d'acceptation.

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM

Semaine 2 - Jour 7 : Construire ton propre cadre d'assurance qualité Agile pour ton équipe

Passer de la théorie à ton cadre qualité personnalisé

Tu as maintenant exploré les valeurs Agile, Scrum, Kanban, SAFe, la DoD/DoR et les critères d'acceptation. Il est temps de synthétiser ces apprentissages en un cadre d'assurance qualité Agile personnalisé - adapté à ton équipe, ton contexte et tes objectifs.

Les composantes d'un cadre QA Agile

Composante	Description	Questions de définition
Valeurs qualité	Les croyances fondamentales qui guident les décisions qualité de l'équipe	Qu'est-ce que signifie la qualité pour nous ? Quels sont nos engagements non-négociables ?
Standards	DoD, DoR, standards de codage, conventions de nommage des tests	Quels sont nos critères d'entrée et de sortie à chaque étape ?
Pratiques	TDD, BDD, pair testing, revues de code, sessions de test exploratoire	Quelles pratiques adoptons-nous systématiquement pour assurer la qualité ?
Outils	Frameworks de test automatisé, outils de gestion de défauts, pipelines CI	Quels outils supportent nos pratiques qualité de façon efficiente ?
Métriques	Indicateurs de qualité suivis et analysés régulièrement	Comment mesurons-nous la qualité de façon objective et continue ?

Composante	Description	Questions de définition
Amélioration continue	Processus de revue et d'amélioration des pratiques qualité	Comment apprenons-nous de nos erreurs et améliorons-nous notre qualité ?

Les métriques qualité Agile à suivre

- **Taux de défauts par sprint :** nombre de défauts trouvés par sprint - tendance et distribution
- **Taux d'échappement des défauts :** pourcentage de défauts trouvés en production vs. en test - le plus critique
- **Couverture de tests automatisés :** pourcentage du code couvert par des tests automatisés
- **Temps moyen de résolution des défauts :** efficacité du processus de correction
- **Vélocité qualité :** items complétant la DoD vs. items démarrés
- **Dette technique :** mesure et tendance de la dette accumulée

Exercice pratique - Semaine 2, Jour 7

Rédige la première version de ton cadre d'assurance qualité Agile pour ton équipe. Inclus : tes 3 valeurs qualité, tes standards (DoD et DoR), tes 5 pratiques clés, tes 3 outils principaux et tes 5 métriques de suivi. Fais-le tenir sur une seule page - la concision est une vertu qualité. Ce document sera ton point de départ pour une conversation avec ton équipe lors de la prochaine rétrospective.

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM

EXERCICE DE TOTALISATION - SEMAINE 2

TTLSTˆ - Semaine 2

Dimension 1 - Substantif linguistique

« Action d'additionner ; action de considérer l'ensemble, le total, de synthétiser. »

Dimension 2 - Philosophie ORIGINALIUM

« Rassembler et réaliser totalement dans son ensemble - avant, pendant et après - en continu, sans interruption. »

Synthétiser comment l'agilité et la qualité se renforcent mutuellement

Cette semaine, tu as exploré comment l'Agile n'est pas simplement une méthode de travail, mais une philosophie qui, lorsqu'elle est vraiment incarnée, renforce naturellement la qualité logicielle. La totalisation de cette semaine consiste à rassembler tous ces apprentissages pour voir l'image complète - comment l'agilité et la qualité sont deux expressions de la même valeur fondamentale : livrer de la valeur réelle aux utilisateurs, en continu, de façon fiable.

Questions de totalisation - Semaine 2

18. **Totalise l'Agile-Qualité :** En quoi les valeurs Agile sont-elles fondamentalement des valeurs qualité ? Quelle connexion profonde vois-tu entre l'adaptation continue et l'excellence qualité ?

19. **Totalise les cadres :** Comment Scrum, Kanban et SAFe se complètent-ils pour assurer la qualité à différentes échelles ? Dans quel contexte utiliserais-tu chacun ?
20. **Totalise les standards :** Comment la DoD et la DoR, les critères d'acceptation et le backlog refinement forment-ils ensemble un système cohérent de protection de la qualité ?
21. **Totalise ton impact :** Quelles pratiques Agile-qualité as-tu l'intention d'introduire ou de renforcer dans ton équipe dans les 30 prochains jours ?
22. **Totalise le tout :** Comment les apprentissages de la Semaine 1 (SDLC) et de la Semaine 2 (Agile) se connectent-ils ? Quelle est la vision unifiée qui émerge ?

Engagement de totalisation - Semaine 2

Complète cette phrase : « La pratique Agile que j'engage à mettre en place ou à améliorer dès maintenant pour renforcer la qualité dans mon équipe est... parce que... et l'impact que j'anticipe est... »

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM

SEMAINE 3 - DevOps, CI/CD et la qualité continue

DevOps a transformé la façon dont les organisations créent, livrent et maintiennent leurs logiciels. Pour le praticien qualité, c'est une révolution qui place la qualité au coeur du pipeline de livraison - automatisée, mesurée et continue. Cette semaine, tu vas maîtriser les concepts et les pratiques DevOps qui font de toi un acteur central de l'excellence logicielle.

Objectifs de la semaine 3

- Comprendre la culture DevOps et son impact sur la responsabilité qualité partagée
- Maîtriser les principes et la mise en oeuvre du CI/CD orienté qualité
- Intégrer la sécurité (DevSecOps) et l'Infrastructure as Code dans la stratégie qualité
- Mesurer la performance de la livraison avec les DORA Metrics
- Concevoir un pipeline CI/CD idéal avec intégration qualité complète

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM

Semaine 3 - Jour 1 : Culture DevOps - collaboration dev-ops-QA, brise les silos, partage de la responsabilité qualité

DevOps : quand la culture précède les outils

DevOps est souvent présenté comme un ensemble d'outils - pipelines CI/CD, Kubernetes, Terraform. Mais avant d'être une boîte à outils, DevOps est une culture. C'est une philosophie de collaboration, de responsabilité partagée et d'amélioration continue qui brise les silos organisationnels qui ont longtemps séparé développeurs, opérations et équipes qualité.

Les silos traditionnels et leur impact sur la qualité

Silo	Mentalité traditionnelle	Impact sur la qualité
Développeurs	« Mon code fonctionne sur ma machine »	Défauts d'environnement, intégration tardive, dette technique
Équipe QA	« Ce n'est pas de notre faute, c'est le code des développeurs »	Qualité perçue comme une étape finale plutôt qu'une responsabilité partagée
Opérations	« La stabilité avant les nouvelles fonctionnalités »	Déploiements rares et risqués, faible fréquence de livraison

Le modèle CALMS - Les piliers de la culture DevOps

- **C - Culture :** Collaboration, confiance et responsabilité partagée entre Dev, Ops et QA. La qualité appartient à tous.
- **A - Automation :** Automatiser les tâches répétitives pour libérer l'équipe pour des tâches à haute valeur - et pour réduire les erreurs humaines.
- **L - Lean :** Éliminer les gaspillages et les délais inutiles dans le flux de livraison, y compris les délais qualité.
- **M - Measurement :** Mesurer tout - performances, qualité, vitesse de livraison - pour prendre des décisions basées sur les faits.
- **S - Sharing :** Partager les connaissances, les outils, les échecs et les succès entre toutes les équipes.

Le rôle du praticien qualité dans la culture DevOps

Dans une culture DevOps mature, le praticien qualité n'est plus le dernier rempart avant la production - il est un partenaire de confiance qui intègre la qualité dans chaque étape du pipeline. Son rôle évolue :

- De : Testeur isolé qui valide en fin de cycle
- Vers : Quality Coach qui guide, automatise, mesure et améliore en continu

Questions de réflexion - Semaine 3, Jour 1

1. Quels silos existent encore dans ton organisation entre Dev, QA et Ops ? Comment affectent-ils la qualité ?
2. Comment la culture de ton équipe se situe-t-elle par rapport au modèle CALMS ? Quel pilier est le plus faible ?
3. Comment ton rôle de praticien qualité a-t-il évolué (ou doit-il évoluer) dans une culture DevOps ?
4. Quelle action concrète pourrais-tu prendre dès demain pour briser un silo qui nuit à la qualité dans ton organisation ?

Exercice pratique - Semaine 3, Jour 1

Effectue une auto-évaluation de la culture DevOps de ton équipe selon le modèle CALMS. Pour chaque pilier, attribue un score de 1 (très faible) à 5 (excellent) et justifie ton évaluation avec des exemples concrets. Identifie le pilier le plus critique à améliorer et propose trois actions concrètes pour l'améliorer dans les 60 prochains jours.

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM

Semaine 3 - Jour 2 : Intégration Continue (CI) - pipelines, tests automatiques, qualité des builds

L'Intégration Continue - le premier pilier de la qualité automatisée

L'Intégration Continue (CI - Continuous Integration) est la pratique qui consiste à intégrer les changements de code de tous les développeurs dans un dépôt partagé plusieurs fois par jour, et à valider automatiquement chaque intégration par un pipeline de vérification. C'est le fondement de la qualité continue dans DevOps.

Les principes fondamentaux de la CI

- Chaque développeur intègre son code dans la branche principale au moins une fois par jour
- Chaque intégration est validée par un build automatisé et une suite de tests
- Si le build échoue, le corriger est la priorité absolue de l'équipe
- Le processus de build doit être rapide (objectif : moins de 10 minutes)
- Les tests doivent s'exécuter dans un environnement représentatif de la production

La pyramide de tests dans le pipeline CI

Niveau de test	Proportion recommandée	Vitesse d'exécution	Rôle dans la CI
Tests unitaires	70% de la suite	Secondes	Validation immédiate de la

Niveau de test	Proportion recommandée	Vitesse d'exécution	Rôle dans la CI
			logique métier - premier garde-fou qualité
Tests d'intégration	20% de la suite	Minutes	Validation des interactions entre composants et services
Tests end-to-end	10% de la suite	Minutes à heures	Validation des flux utilisateur critiques - sélection rigoureuse nécessaire

Les étapes d'un pipeline CI orienté qualité

23. **Source :** Commit et push du développeur sur la branche de travail
24. **Build :** Compilation du code - vérification de la syntaxe et des dépendances
25. **Analyse statique :** Linting, formatage, analyse de code (SonarQube, ESLint, Checkstyle)
26. **Tests unitaires :** Exécution de la suite complète de tests unitaires avec rapport de couverture
27. **Tests d'intégration :** Validation des interfaces et des interactions entre composants
28. **Rapport de qualité :** Tableau de bord des métriques qualité du build (couverture, dettes, violations)
29. **Notification :** Alerte immédiate à l'équipe si le build ou un test échoue

Quality Gates - les portes de qualité automatisées

Les Quality Gates sont des seuils de qualité configurés dans ton pipeline CI. Si une métrique tombe en dessous du seuil défini, le pipeline s'arrête et le code ne peut pas progresser vers l'étape suivante. Exemples de Quality Gates efficaces :

- Couverture de code : minimum 80% - en dessous, le build échoue

- Zéro nouvelle violation critique de sécurité introduite
- Score de maintenabilité SonarQube : pas de dégradation du rating
- Aucun test de régression en échec
- Duplication de code : pas d'augmentation au-delà du seuil défini

Questions de réflexion - Semaine 3, Jour 2

1. Ton équipe pratique-t-elle l'Intégration Continue ? À quelle fréquence les développeurs intègrent-ils leur code ?
2. Quels types de tests sont automatisés dans ton pipeline CI actuel ? Quelle est ta pyramide de tests réelle ?
3. As-tu des Quality Gates configurés ? Sont-ils respectés ou contournés sous la pression ?
4. Quel est le temps d'exécution de ton pipeline CI actuel ? Est-ce un obstacle à l'adoption de la CI rapide ?

Exercice pratique - Semaine 3, Jour 2

Documente ton pipeline CI actuel étape par étape. Pour chaque étape, note : (1) est-elle présente dans ton pipeline ?, (2) est-elle automatisée ?, (3) quelle est sa durée d'exécution ?, (4) quelles métriques qualité génère-t-elle ? Identifie les trois lacunes les plus critiques et propose comment les combler.

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM

Semaine 3 - Jour 3 : Livraison Continue (CD) - déploiement automatisé, tests de smoke, gates de qualité

La Livraison Continue - automatiser la confiance jusqu'en production

La Livraison Continue (CD - Continuous Delivery ou Continuous Deployment) est l'extension naturelle de la CI. Là où la CI valide que le code est de qualité, la CD assure que ce code peut être livré en production à tout moment - de façon fiable, reproductible et avec un minimum de risque.

Continuous Delivery vs. Continuous Deployment - une distinction importante

Pratique	Description	Niveau d'automatisation
Continuous Delivery	Le code est toujours dans un état déployable. Le déploiement en production nécessite une approbation humaine manuelle.	Automatisé jusqu'à la porte de production - décision humaine pour le déclenchement
Continuous Deployment	Chaque changement validé par le pipeline est automatiquement déployé en production sans intervention humaine.	Entièrement automatisé de bout en bout

Les étapes d'un pipeline CD orienté qualité

30. **Déploiement en environnement de test :** Déploiement automatisé et reproductible dans un environnement identique à la production

31. **Tests de smoke :** Validation rapide (5 à 10 minutes) que les fonctions vitales du système fonctionnent après le déploiement
32. **Tests fonctionnels automatisés :** Exécution de la suite de tests de régression complète dans l'environnement déployé
33. **Tests de performance :** Validation que les critères de performance sont respectés
34. **Tests de sécurité dynamiques (DAST) :** Analyse de sécurité en conditions réelles
35. **Quality Gate de déploiement :** Validation de toutes les métriques avant promotion à l'étape suivante
36. **Déploiement en staging/pré-production :** Dernière validation avant production
37. **Tests d'acceptation utilisateur (UAT) :** Validation humaine si requis
38. **Déploiement en production :** Automatisé ou déclenché manuellement selon la stratégie

Les tests de smoke - ta première ligne de défense post-déploiement

Les tests de smoke (ou sanity tests) sont des tests légers et rapides exécutés immédiatement après chaque déploiement pour vérifier que les fonctions les plus critiques du système fonctionnent. Ils ne sont pas exhaustifs - ils sont stratégiques : si les fonctions vitales sont opérationnelles, le déploiement est considéré stable.

Caractéristiques des bons tests de smoke

- Rapides - exécution en moins de 10 minutes
- Ciblés sur les fonctions critiques (authentification, paiement, flux principal)
- Totalement automatisés et fiables
- Indépendants des données de test - utilisent des données fixes ou

contrôlées

- Déclenchent automatiquement un rollback si l'un d'eux échoue

Questions de réflexion - Semaine 3, Jour 3

1. Ton organisation pratique-t-elle la Continuous Delivery ou le Continuous Deployment ? Quelle est la fréquence de tes déploiements en production ?
2. As-tu des tests de smoke automatisés après chaque déploiement ? Si oui, couvrent-ils vraiment les fonctions vitales ?
3. Quelle est ta stratégie de rollback actuelle si un déploiement introduit une régression critique ?
4. Quels sont les principaux obstacles à l'adoption d'un vrai pipeline CD dans ton organisation ?

Exercice pratique - Semaine 3, Jour 3

Identifie les 10 fonctions les plus critiques de ton application (ou d'une application que tu connais bien). Pour chacune, rédige un cas de test de smoke sous la forme : « ÉTANT DONNÉ [état initial], QUAND [action de validation], ALORS [résultat attendu]. » Ces 10 tests devraient constituer ta suite de smoke minimale après chaque déploiement.

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM

Semaine 3 - Jour 4 : Infrastructure as Code et qualité - tests d'infrastructure, Terraform, Ansible, validation

Appliquer la qualité logicielle à l'infrastructure elle-même

L'Infrastructure as Code (IaC) est la pratique qui consiste à gérer et à provisionner l'infrastructure (serveurs, réseaux, bases de données, configurations) via du code plutôt que des processus manuels. Pour le praticien qualité, l'IaC est une révolution : l'infrastructure devient testable, reproductible et versionnable - comme le code applicatif.

Pourquoi l'IaC est une victoire pour la qualité

- **Reproductibilité :** Chaque environnement (dev, test, staging, prod) est identique - les bugs « ça marche sur ma machine » disparaissent
- **Versionning :** L'infrastructure est versionnée dans Git - on peut voir qui a changé quoi et quand
- **Testabilité :** L'infrastructure peut être testée avant d'être déployée
- **Déploiements cohérents :** Pas d'erreur humaine dans la configuration manuelle des serveurs
- **Rollback simplifié :** Revenir à une version précédente de l'infrastructure est aussi simple que de revert un commit

Les outils IaC principaux et leur angle qualité

Outil	Usage principal	Tests qualité associés
Terraform	Provisionner l'infrastructure cloud	terraform validate, terraform plan review,

Outil	Usage principal	Tests qualité associés
	(AWS, Azure, GCP) en code déclaratif	tests avec Terratest, Checkov pour la sécurité IaC
Ansible	Configuration et déploiement des serveurs via des playbooks	Molecule pour tester les playbooks, yamllint, ansible-lint
Docker / Kubernetes	Conteneurisation et orchestration des applications	Hadolint pour Dockerfile, Trivy pour les images, Kube-bench pour la sécurité Kubernetes
CloudFormation / ARM	Templates IaC natifs AWS et Azure	cfn-lint, cfn-nag pour la sécurité, AWS Config Rules pour la conformité

Intégrer les tests d'infrastructure dans le pipeline CI/CD

Les tests d'infrastructure doivent être intégrés dans le pipeline avec le même niveau de rigueur que les tests applicatifs :

39. **Linting et validation syntaxique :** Vérification automatique de la syntaxe des fichiers IaC à chaque commit
40. **Analyse de sécurité statique :** Détection des mauvaises pratiques de sécurité (ports ouverts, ressources non chiffrées)
41. **Plan review :** Revue automatisée et humaine du plan de changement avant application
42. **Tests fonctionnels d'infrastructure :** Déploiement dans un environnement temporaire et validation que l'infrastructure se comporte comme attendu
43. **Tests de conformité :** Validation que l'infrastructure respecte les politiques de sécurité et de gouvernance
44. **Tests de destruction :** Validation que l'infrastructure peut être détruite et recréée proprement (chaos engineering léger)

Questions de réflexion - Semaine 3, Jour 4

1. Ton organisation utilise-t-elle l'IaC ? À quel niveau de maturité ? Est-ce que toute l'infrastructure est codifiée ou seulement une partie ?
2. As-tu déjà vécu un incident en production causé par une différence de configuration entre environnements ? L'IaC l'aurait-il évité ?
3. Qui est responsable de la qualité de l'infrastructure dans ton organisation ? Est-ce que l'équipe QA est impliquée ?
4. Quelles pratiques de test d'infrastructure pourrais-tu introduire dès maintenant dans ton pipeline ?

Exercice pratique - Semaine 3, Jour 4

Définis une stratégie de test d'infrastructure pour un composant critique de ton système (base de données, serveur web, load balancer, etc.). Décris : (1) quels outils IaC tu utiliserais, (2) quels types de tests tu effectuerais à chaque étape, (3) comment tu intègrerais ces tests dans le pipeline CI/CD, (4) quels critères de qualité de l'infrastructure tu définirais.

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM

Semaine 3 - Jour 5 : Sécurité dans le pipeline (DevSecOps) - SAST, DAST, analyse de dépendances

DevSecOps - quand la sécurité devient une responsabilité qualité partagée

DevSecOps est l'évolution naturelle de DevOps qui intègre la sécurité (Security) dès le début du cycle de développement - plutôt que de la traiter comme une vérification finale ou une responsabilité exclusive d'une équipe spécialisée. Pour le praticien qualité, c'est une extension directe du Shift Left : la sécurité devient une dimension de la qualité, intégrée dans chaque étape du pipeline.

Le Shift Left de la sécurité - Security as Code

L'approche traditionnelle de la sécurité attendait la fin du développement pour effectuer un test de pénétration (pentest). DevSecOps déplace ces contrôles tout au long du pipeline :

Étape du pipeline	Contrôle de sécurité intégré	Outil type
Développement (IDE)	Analyse statique en temps réel, suggestions de corrections sécurité	SonarLint, Snyk IDE plugins
Commit / CI	SAST - Analyse statique du code pour détecter les vulnérabilités	SonarQube, Checkmarx, Semgrep, Bandit (Python)
CI - Dépendances	SCA - Analyse des dépendances et librairies tierces (CVE)	Snyk, OWASP Dependency-Check, Dependabot
Build - Images	Scan des images Docker pour les vulnérabilités	Trivy, Clair, Grype
CD - Tests	DAST - Analyse dynamique de	OWASP ZAP, Burp Suite, Nikto

Étape du pipeline	Contrôle de sécurité intégré	Outil type
	l'application déployée	
Production	RASP - Protection en temps réel, monitoring sécurité	Contrast Security, Datadog Security Monitoring

SAST - Static Application Security Testing

Le SAST analyse le code source sans l'exécuter pour détecter des patterns de vulnérabilités connus. Il est particulièrement efficace pour détecter :

- Injections SQL et NoSQL
- Cross-Site Scripting (XSS)
- Mauvaise gestion des secrets et des credentials dans le code
- Désérialisation non sécurisée
- Mauvaise gestion des erreurs qui expose des informations sensibles

DAST - Dynamic Application Security Testing

Le DAST analyse l'application en cours d'exécution en simulant des attaques réelles. Il détecte des vulnérabilités qui n'apparaissent qu'au runtime :

- Problèmes de configuration serveur
- Vulnérabilités d'authentification et de gestion de session
- Exposition d'informations sensibles dans les réponses API
- Failles d'autorisation (accès à des ressources non autorisées)
- Vulnérabilités liées à l'environnement de déploiement

SCA - Software Composition Analysis

Plus de 80% des applications modernes sont constituées de code open source et de bibliothèques tierces. Le SCA analyse ces dépendances pour détecter des vulnérabilités connues (CVE - Common Vulnerabilities and Exposures) et des problèmes de licence.

Questions de réflexion - Semaine 3, Jour 5

1. Quels contrôles de sécurité automatisés sont actuellement intégrés dans ton pipeline CI/CD ? Sont-ils suffisants ?
2. As-tu déjà trouvé une vulnérabilité de sécurité lors de tes tests qui aurait pu être détectée plus tôt avec du SAST ?
3. Comment est gérée la sécurité des dépendances dans ton projet ? Utilises-tu des outils de SCA ?
4. Comment la sécurité est-elle partagée entre l'équipe de développement, QA et la sécurité dans ton organisation ?

Exercice pratique - Semaine 3, Jour 5

Conçois une stratégie DevSecOps pour ton pipeline CI/CD. Pour chaque étape du pipeline, identifie : (1) quel type de contrôle de sécurité serait intégré (SAST, DAST, SCA, autre), (2) quel outil tu utiliserais, (3) comment les résultats seraient traités (bloquant, avertissement, rapport), (4) qui est responsable de la correction. Priorise selon la criticité de ton contexte.

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM

Semaine 3 - Jour 6 : Mesures et métriques DevOps - DORA Metrics

Mesurer la performance de la livraison pour piloter la qualité

Les DORA Metrics (DevOps Research and Assessment) sont les quatre métriques clés qui mesurent la performance de la livraison logicielle et la santé organisationnelle d'une équipe DevOps. Développées par le programme DORA de Google et popularisées dans le livre « Accelerate » de Nicole Forsgren, Jez Humble et Gene Kim, elles sont devenues le standard de l'industrie pour évaluer l'efficacité du pipeline de livraison.

Les 4 DORA Metrics expliquées

Métrique	Définition	Ce qu'elle révèle sur la qualité
Deployment Frequency (DF)	Fréquence à laquelle le code est déployé en production	Une haute fréquence indique un pipeline fiable et une qualité intégrée. Une faible fréquence signale des lots de changement trop grands et des risques qualité élevés.
Lead Time for Changes (LTC)	Temps entre un commit de code et son déploiement réussi en production	Un Lead Time court indique un processus de validation qualité efficace. Un Lead Time long révèle des goulots d'étranglement dans le pipeline de qualité.
Change Failure Rate (CFR)	Pourcentage des déploiements qui causent un incident ou une dégradation en production	Indicateur direct de la qualité du code et du pipeline de validation. Un CFR élevé révèle des lacunes dans la stratégie de tests avant

Métrique	Définition	Ce qu'elle révèle sur la qualité
		production.
Mean Time to Recovery (MTTR)	Temps moyen pour restaurer le service après un incident de production	Un MTTR court indique de bonnes pratiques de monitoring, de diagnostique et de rollback - tous des aspects de la qualité opérationnelle.

Les niveaux de performance DORA

Métrique	Elite	Haute	Moyenne	Faible
Deployment Frequency	Plusieurs fois/jour	1 fois/jour à 1 fois/semaine	1 fois/semaine à 1 fois/mois	Moins d'une fois/mois
Lead Time for Changes	Moins d'1 heure	1 jour à 1 semaine	1 semaine à 1 mois	Plus d'1 mois
Change Failure Rate	0% à 15%	0% à 15%	0% à 15%	Plus de 15%
MTTR	Moins d'1 heure	Moins d'1 jour	1 jour à 1 semaine	Plus d'1 semaine

Utiliser les DORA Metrics pour piloter l'amélioration qualité

- **CFR élevé :** Renforcer les tests avant production, améliorer les tests de régression, revoir les Quality Gates
- **MTTR élevé :** Améliorer le monitoring, renforcer les tests de smoke, automatiser les rollbacks
- **Lead Time élevé :** Identifier les goulots dans le pipeline, paralléliser les tests, optimiser la suite de tests
- **Deployment Frequency faible :** Réduire la taille des lots, améliorer la confiance dans le pipeline, renforcer l'automatisation

Questions de réflexion - Semaine 3, Jour 6

1. Connais-tu tes métriques DORA actuelles ? Si non, pourquoi penses-tu qu'elles ne sont pas mesurées ?
2. Laquelle des 4 métriques DORA est la plus critique à améliorer dans ton contexte actuel ? Pourquoi ?
3. Comment est-ce que le Change Failure Rate de ton équipe se compare aux benchmarks de l'industrie ? Qu'est-ce que cela te dit sur la maturité de ton pipeline qualité ?
4. Quels liens vois-tu entre tes pratiques de test actuelles et tes métriques DORA (ou leur absence) ?

Exercice pratique - Semaine 3, Jour 6

Mesure ou estime tes 4 métriques DORA actuelles. Positionne-toi dans les niveaux de performance (Elite, Haute, Moyenne, Faible) pour chaque métrique. Identifie la métrique qui a le plus grand impact sur la qualité de ton produit et rédige un plan d'amélioration en 90 jours avec des étapes concrètes et des indicateurs de succès.

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM

Semaine 3 - Jour 7 : Concevoir ton pipeline CI/CD idéal avec intégration qualité complète

Synthétiser trois semaines en ton pipeline qualité de référence

Tu arrives au dernier jour de la Partie A. Cette journée de synthèse est l'occasion de rassembler tous tes apprentissages en un pipeline CI/CD idéal qui intègre la qualité à chaque étape - du commit au monitoring en production.

Les dimensions d'un pipeline CI/CD qualité complet

Dimension	Éléments clés
Qualité du code	Linting, analyse statique, couverture de tests, revues de code automatisées
Qualité fonctionnelle	Tests unitaires, tests d'intégration, tests end-to-end, tests de régression
Qualité de sécurité	SAST, SCA, scan d'images, DAST, tests de conformité
Qualité de performance	Tests de charge, tests de stress, tests de scalabilité
Qualité d'infrastructure	Validation IaC, tests d'infrastructure, tests de configuration
Qualité opérationnelle	Tests de smoke post-déploiement, monitoring, alertes, DORA Metrics

Architecture d'un pipeline CI/CD idéal

45. **Source :** Commit sur branche feature - revue de code pair obligatoire pour merge
46. **Build :** Compilation, résolution des dépendances, création de l'artefact
47. **Analyse statique :** SAST, linting, analyse de code (SonarQube), SCA

48. **Tests unitaires :** Suite complète avec rapport de couverture - Quality Gate 80%+
49. **Tests d'intégration :** Validation des interfaces et des services
50. **Scan de sécurité des images :** Trivy ou équivalent sur l'image Docker créée
51. **Déploiement en test :** Déploiement automatisé via IaC dans un environnement identique à la prod
52. **Tests de smoke :** Validation rapide des fonctions vitales post-déploiement
53. **Tests fonctionnels :** Suite de régression complète - tests automatisés BDD
54. **Tests de performance :** Validation des critères de performance définis
55. **DAST :** Analyse de sécurité dynamique de l'application déployée
56. **Quality Gate finale :** Validation de toutes les métriques avant promotion
57. **Déploiement staging :** Validation UAT si requise
58. **Déploiement production :** Automatisé avec feature flags et canary release
59. **Monitoring post-déploiement :** Alertes automatiques, DORA Metrics, dashboard qualité

Exercice pratique - Semaine 3, Jour 7 : Ton pipeline idéal

Conçois ton pipeline CI/CD idéal pour ton contexte spécifique. Pour chaque étape que tu retiens, définis : (1) l'objectif qualité de l'étape, (2) l'outil ou la pratique utilisée, (3) les critères de succès (Quality Gate), (4) l'action en cas d'échec. Identifie aussi les étapes qui sont actuellement absentes de ton pipeline et priorise leur introduction. C'est ton plan de transformation qualité DevOps.

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM

EXERCICE DE TOTALISATION - SEMAINE 3

ŢŢĻŞŢ~ - Semaine 3

Dimension 1 - Substantif linguistique

« Action d'additionner ; action de considérer l'ensemble, le total, de synthétiser. »

Dimension 2 - Philosophie ORIGINALIUM

« Rassembler et réaliser totalement dans son ensemble - avant, pendant et après - en continu, sans interruption. »

Totaliser comment DevOps et la qualité continue transforment le SDLC

DevOps et la qualité continue ne sont pas des ajouts au SDLC - ils sont sa transformation fondamentale. Là où le SDLC classique séquençait les activités, DevOps les parallélise, les automatise et les rend continues. La totalisation de cette semaine consiste à voir comment tout ce que tu as exploré - culture, CI, CD, IaC, sécurité, métriques - forme un écosystème de qualité vivant et dynamique.

Questions de totalisation - Semaine 3

60. **Totalise DevOps :** En quoi DevOps est-il une révolution culturelle autant que technique ? Comment cette culture change-t-elle ta façon de concevoir ton rôle de praticien qualité ?
61. **Totalise le pipeline :** Comment CI, CD, IaC, DevSecOps et les DORA Metrics forment-ils ensemble un système cohérent de livraison de qualité continue ? Quelle est la connexion entre chacun ?
62. **Totalise la sécurité :** Pourquoi la sécurité est-elle une dimension de la qualité et non un domaine séparé ? Comment DevSecOps change-t-il ta responsabilité en tant que praticien qualité ?
63. **Totalise les métriques :** Comment les DORA Metrics te donnent-elles une vision objective de la santé de ton pipeline qualité ? Quelle décision d'amélioration te donnent-elles le courage de prendre ?
64. **Totalise les trois semaines :** Du SDLC fondamental (Semaine 1) à l'Agile (Semaine 2) puis à DevOps (Semaine 3) - quelle est la trajectoire de transformation qualité qui se dessine ? Où en es-tu dans cette trajectoire ?

Engagement de totalisation - Semaine 3

Écris ta vision personnelle de l'excellence qualité dans un contexte DevOps. En 10 à 15 lignes maximum, décris : ce que « qualité continue » signifie pour toi, comment tu incarnes cette vision dans ton quotidien, et ce que tu vas changer concrètement dans les 90 prochains jours pour te rapprocher de cette vision.

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM

Pont vers la Partie B

Ce qui t'attend dans la Partie B - Semaines 4 à 6

Tu as accompli quelque chose de remarquable dans cette Partie A. Tu as posé les bases conceptuelles et pratiques qui forment le socle de toute stratégie qualité dans le développement logiciel moderne. Le SDLC, ses modèles, le Shift Left, les valeurs Agile, Scrum, Kanban, SAFe, la DoD et la DoR, le backlog refinement, la culture DevOps, le CI/CD, l'IaC, DevSecOps et les DORA Metrics - tout cela est maintenant une partie de ta vision et de ton vocabulaire professionnel.

Mais le voyage continue. La Partie B va te plonger dans les phases critiques de développement, de test et de déploiement avec un niveau de profondeur et de pratique encore plus élevé. Tu vas explorer les stratégies de test avancées, les techniques d'automatisation, la gestion des environnements de test, les tests de performance et de sécurité en profondeur, et les pratiques de livraison qui distinguent les équipes d'élite.

Si la Partie A t'a appris à voir le SDLC comme un écosystème de qualité, la Partie B va t'apprendre à le maîtriser avec l'expertise d'un praticien de haut niveau. Continue.

- Léo Michel Bélanger

TTLST~

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM

Mes notes - Page 1

Cet espace t'appartient. Utilise-le pour noter tes réflexions, tes idées, tes questions et tes engagements au fil de ta lecture.

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM

Mes notes - Page 2

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM

Mes notes - Page 3

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM

TTLSTT~

TTLSTT~

Livre L3 - Partie A

Réconciliation avec le cycle de vie de la réalisation du logiciel (SDLC)

© Léo Michel Bélanger - Tous droits réservés

Images et couverture par Soléane Bélanger

ORIGINALIUM

BEASTEST ORIGINALIUM - Collection complète disponible sur originalium.com

---

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM